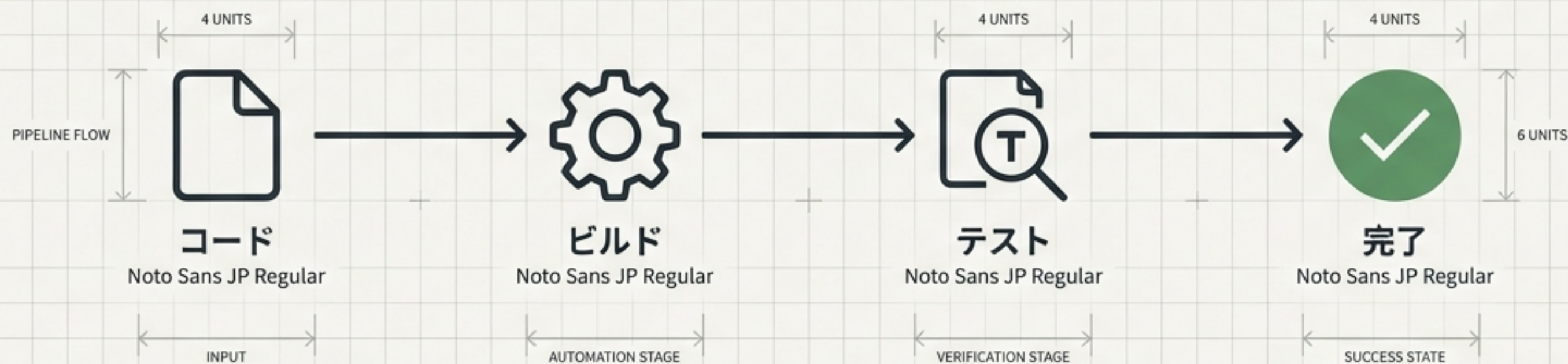
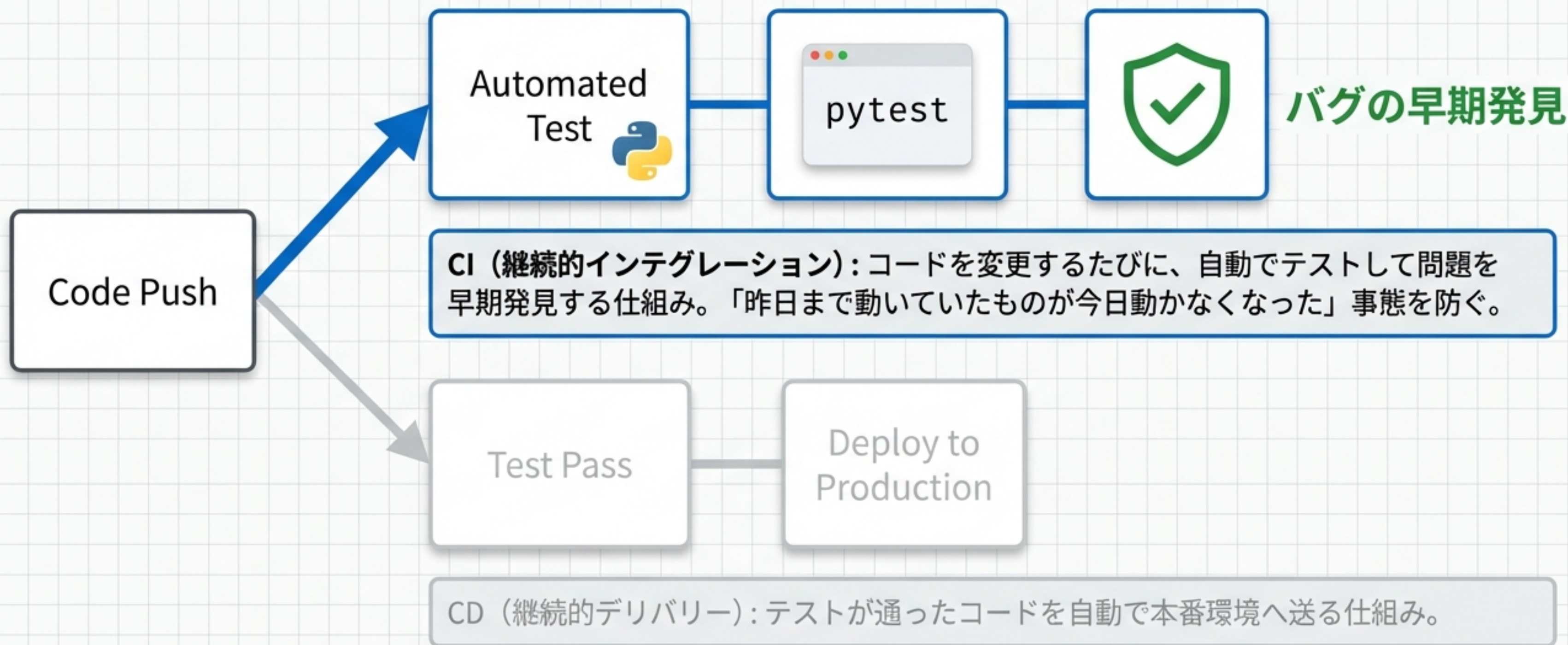


GitHub Actionsで作るPython テスト自動化の設計図



手動テストの手間を省き、コードの品質を自動で担保する



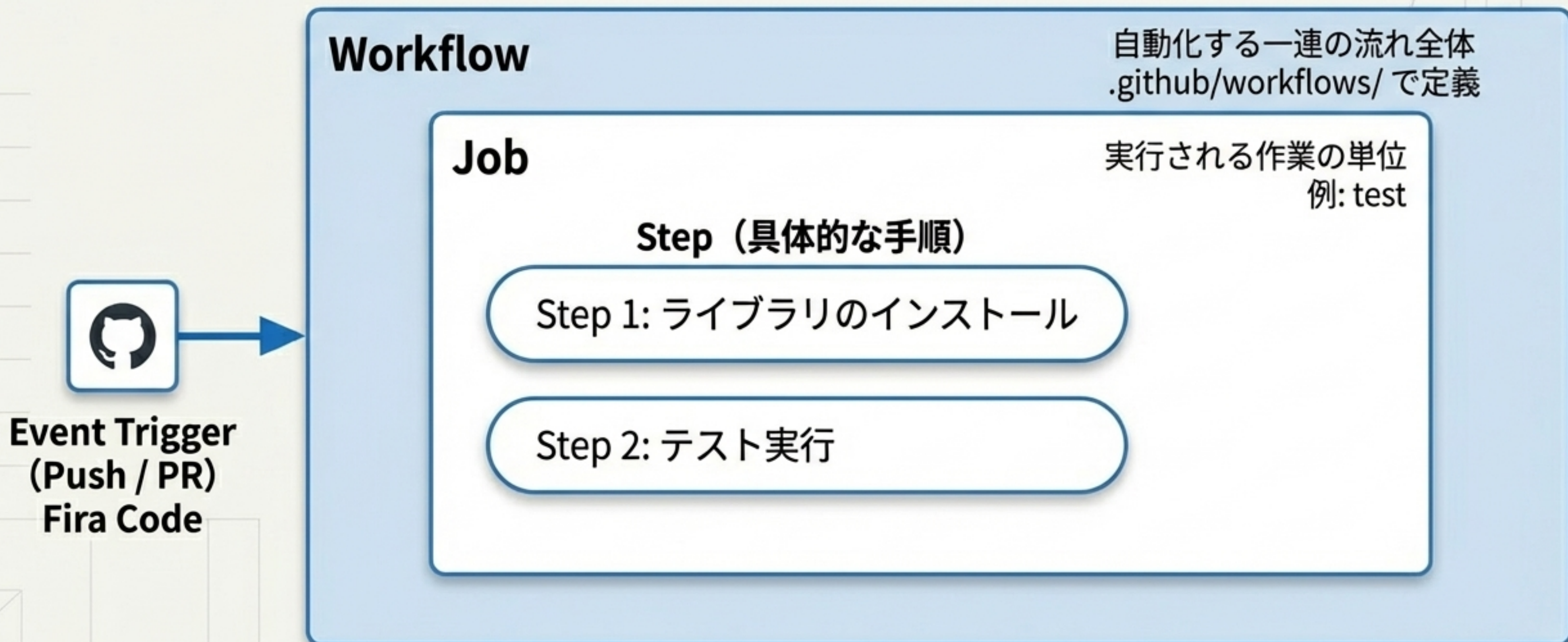
開発ツールをGitHubに集約し、管理コストを最小化する

GitHub上でCI/CDを走らせることで、コード管理から品質保証までを1つのプラットフォームに統合できる。

	従来のCIツール（例: CircleCI）	GitHub Actions
設定場所	外部サイトでの初期設定が必要	✔️ GitHubリポジトリ内で完結（一元管理）
ファイル管理	複数ステップの煩雑な管理	✔️ ジョブと設定ファイルが「1対1」で対応し管理が容易
連携	APIフックの設定が必要	✔️ PRへのコメントやバッジ付与がネイティブでシームレス

自動化のルールは「ワークフロー・ジョブ・ステップ」の3層構造で定義する

GitHub上のイベント（Pushなど）をトリガーに、この入れ子構造のルールが自動実行される。



いつ、どの環境で自動化を起動するかを指定する

```
name: Python Testing CI

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  test:
    runs-on: ubuntu-latest
```

実行タイミング：mainブランチへのPushやPR時に起動

仮想マシン：GitHubが用意する最新のLinux環境を使用

仮想サーバー上に自身のコードとPython環境を準備する

steps:

- name: Check out repository
uses: actions/checkout@v4
- name: Set up Python
uses: actions/setup-python@v5
with:
python-version: '3.10'



実行環境に自身のプログラムコードをコピー（チェックアウト）する



指定したバージョンのPythonをインストールする

必要なライブラリを導入し、自動テストを叩く

- name: Install dependencies

run: |

```
python -m pip install --upgrade pip  
pip install pytest pytest-cov mock
```

テスト実行に必要なライブラリ
群を仮想環境に導入

- name: Run tests

run: `pytest`

ついに本番！
ここで1つでも失敗すると、
GitHub上に✖マークがつく

データベース連携はバックグラウンド起動とデータ投入の順序が鍵になる

アプリケーションがDBに接続する場合、ワークフロー内でDockerコンテナを起動させ、データをセットアップしてからテストを走らせる。



テスト結果とカバレッジをプルリクエスト画面に直接フィードバックする

テスト結果が自動コメントされることで、レビューワーカーの確認コストが大幅に下がり、開発体験（DX）が向上する。



```
uses: MishaKav/pytest-coverage-comment@main  
with:
```

```
  pytest-coverage-path: ./pytest-coverage.txt
```

```
  junitxml-path: ./pytest.xml
```



✓ Tests: 14 passed, 0 failed



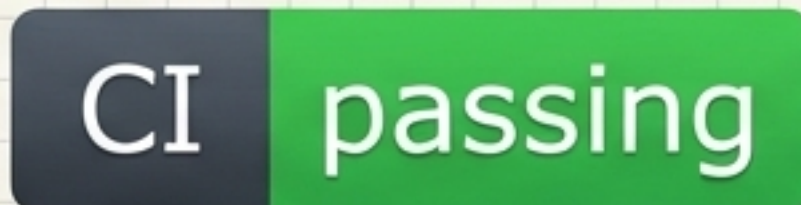
Coverage: 92%

プロジェクトの健全性をREADMEのステータスバッジで証明する

READMEにバッジを貼ることで、テストが通っていることを視覚的に証明。
ジョブごとにバッジを生成し、リンク先を正しいCIのURLに設定することがポイント。

```
[![CI](https://github.com/user/repo/actions/workflows/ci.yml/badge.svg)](https://github.com/user/repo/actions)
```

GitHub Actionsが自動生成するSVG画像を参照する仕組み



ログの最下部にある「サマリー」からエラーの真因を読み解く

赤い✖が出てもパニックにならない。ログの一番下にある short test summary info に答えがある。

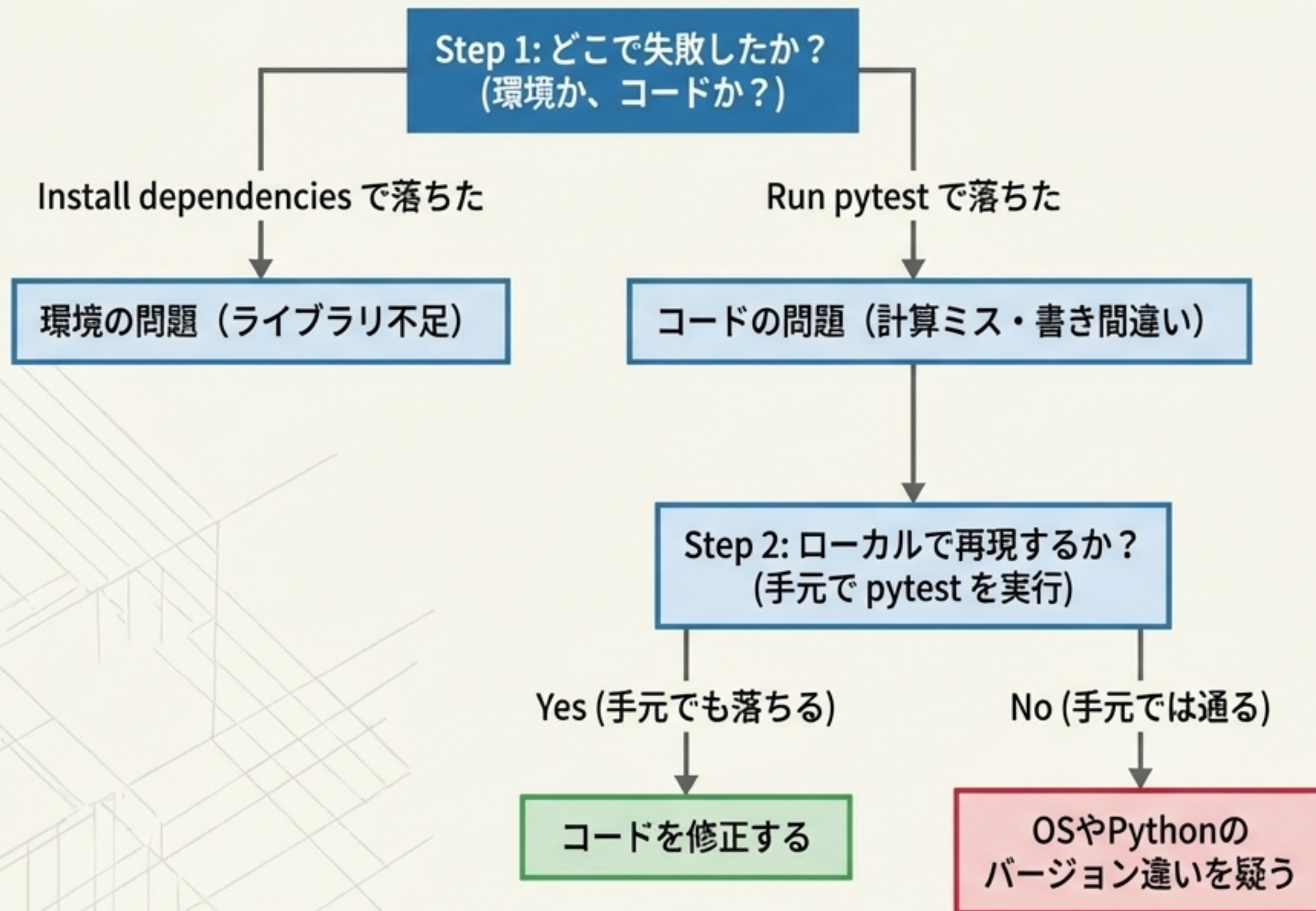
```
===== FAILURES =====
_____ test_addition _____
tests/test_math.py:5: in test_addition
    assert 2 == 3
E   assert 2 == 3
E   +   where 2 = add(1, 1)
```

左辺（実際の結果）と右辺（期待値）の不一致を示す（E = Error）

```
===== short test summary info =====
FAILED tests/test_math.py::test_addition
```

どのファイルの、どのテスト関数が落ちたか

テストが落ちた時のパニックを防ぐ、デバッグの思考プロセス



Pro Tip

Step 3: あえて失敗させる

初心者のうちは、意図的に失敗させて挙動に慣れることが重要。

初学者が必ず直面する代表的なエラーと処方箋

エラータイプ	原因と対策
ModuleNotFoundError	requirements.txt にライブラリを書き忘れている。 必要なパッケージがインストールされていない。
SyntaxError / IndentError	カッコの閉じ忘れや、YAML/Pythonの字下げ（インデント）のズレなど基本的な文法ミス。
AssertionError	一番多いエラー。計算結果が予想と違う。テスト対象のロジックやテストコードの期待値を見直す合図。

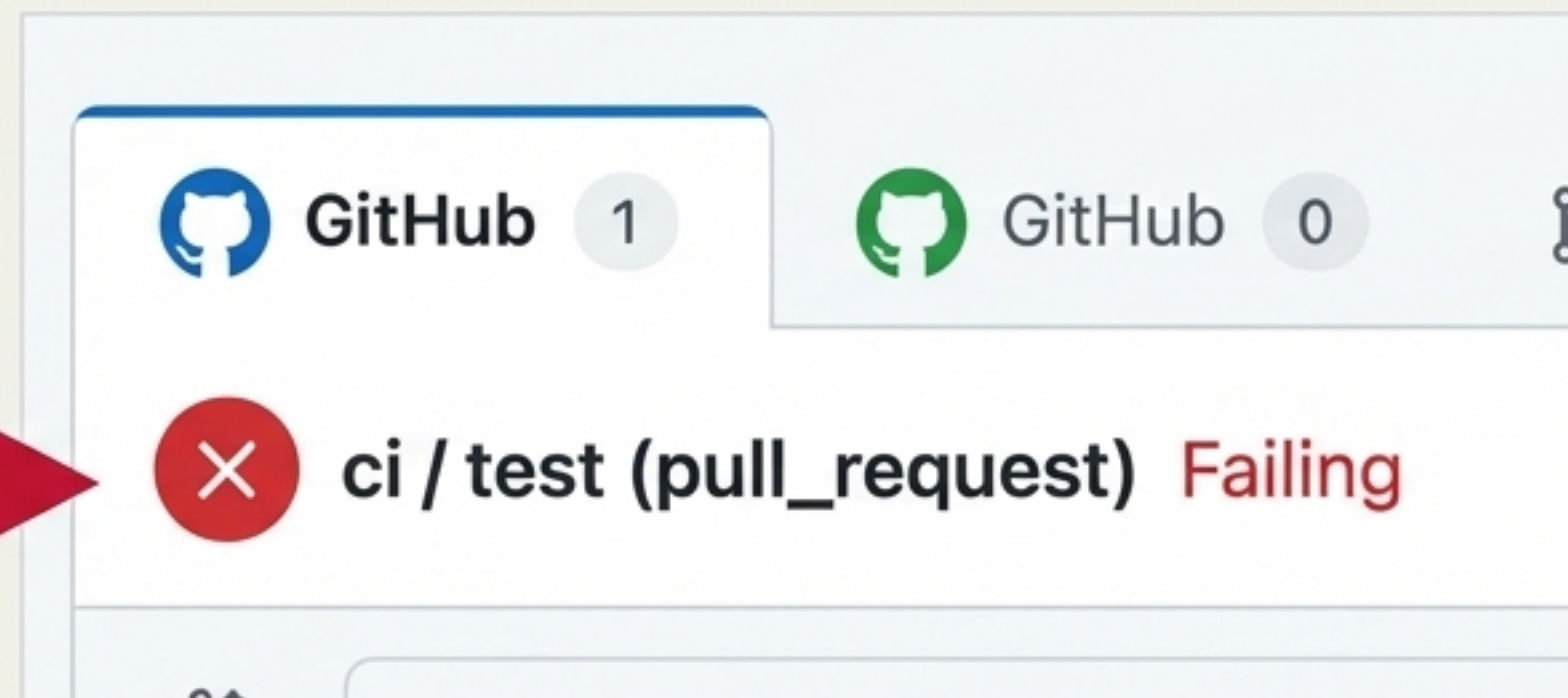
コード変更から品質証明までが全自動で連動するパイプライン

これまで学んだ「トリガー」「環境構築」「テスト実行」「通知」のすべてが連動し、強力な自動テスト環境を形成する。



まずは意図的にテストを失敗させ、システムの挙動を体験する

```
def test_intentional_failure():  
    # わざと失敗させる  
    assert 1 == 2
```



CIの真価は「失敗したときに、いかに早く原因を見つけるか」にある。まずはシンプルな ci.yml 作成し、わざとエラーを起こすことで、この強力なツールに対する恐怖心を無くすことから始めよう。手動テストからの脱却は、すぐそこにある。